0110. Balanced Binary Tree

Easy · Trees

leetcode.com/problems/balanced-binary-tree

Problem

- A tree is height-balanced if, at every node, the left and right subtree heights differ by at most 1. Check it.

Intuition

- Compute heights bottom-up; if any subtree is unbalanced, propagate a sentinel (-1) so we stop early.

Brute Force

- At every node call a separate height(): O(n^2) for skewed trees.

Optimal Approach

- Check(null) = 0.
- L = check(left), r = check(right). If either is -1 or |l - r| > 1, return -1.
- Else return 1 + max(l, r). Balanced iff check(root) != -1.

Worked Example

```
[1,2,2,3,3,null,null,4,4].

node 3s: height 2 (children 4,4)
left 2: l=2 r=2 -> height 3 ; right 2: height 1
root: |3 - 1| = 2 > 1 -> -1 -> not balanced
```

Java Solution

```java
public class BalancedBinaryTree {
    public boolean isBalanced(TreeNode root) { return check(root) != -1; }

    private int check(TreeNode n) {
        if (n == null) return 0;
        int l = check(n.left);
        if (l == -1) return -1;
        int r = check(n.right);
        if (r == -1 || Math.abs(l - r) > 1) return -1;
        return 1 + Math.max(l, r);
    }
}
```

Complexity

- Time: O(n). Space: O(h).

Pitfalls

- Checking only the root's balance is not enough; every node must satisfy it.